DBMS – SALES AND CUSTOMER ANALYTICS SYSTEM

Project title – Flipkart E –Commerce Platform Analysis

Skill Task: DBMS

Week: 9

1. Aggregate Functions

Definition

Aggregate functions are SQL functions used to perform calculations on a group of rows and return a single result.

Types of Aggregate Functions in SQL

1. **COUNT()** – Counts the number of rows.
2. **SUM()** – Calculates the total value.
3. **AVG()** – Calculates the average value.
4. **MIN()** – Finds the smallest value.
5. **MAX()** – Finds the largest value.

1.1 COUNT()

Find the total number of orders:

```
SQL> SELECT COUNT(*) AS TOTAL_ORDERS
  2  FROM Orders;

TOTAL_ORDERS
------------
           5
```

1.2 SUM()

Find the total sales amount:

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES
  2  FROM Orders;

TOTAL_SALES
-----------
      84748
```

1.3 AVG()

Find the average order amount:

```
SQL> SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_AMOUNT
  2  FROM Orders;

AVERAGE_ORDER_AMOUNT
--------------------
             16949.6
```

1.4 MIN()

Find the lowest order amount:

```
SQL> SELECT MIN(TOTAL_AMOUNT) AS LOWEST_ORDER_AMOUNT
  2  FROM Orders;

LOWEST_ORDER_AMOUNT
-------------------
                650
```

1.5 MAX()

Find the highest order amount:

```
SQL> SELECT MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER_AMOUNT
  2  FROM Orders;

HIGHEST_ORDER_AMOUNT
--------------------
               55000
```

2. Generate Total Sales Reports

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES
  2  FROM Orders;

TOTAL_SALES
-----------
      84748
```

2.1 Customer-wise Sales Report

To find how much each customer has purchased:

```
SQL> SELECT c.ID AS CUSTOMER_ID,
  2         c.NAME AS CUSTOMER_NAME,
  3         SUM(o.TOTAL_AMOUNT) AS TOTAL_PURCHASE_AMOUNT
  4  FROM Customer c
  5  JOIN Orders o
  6  ON c.ID = o.CUSTOMER_ID
  7  GROUP BY c.ID, c.NAME
  8  ORDER BY TOTAL_PURCHASE_AMOUNT DESC;

CUSTOMER_ID CUSTOMER_NAME        TOTAL_PURCHASE_AMOUNT
----------- -------------------- --------------------
          1 Renu                                55000
          2 Arjun                               25000
          4 Rahul                                2500
          3 Priya                                1598
          5 Keerthana                             650
```

3. Find Top Customers Based on Purchase Amount

```
SQL> SELECT c.ID AS CUSTOMER_ID,
  2         c.NAME AS CUSTOMER_NAME,
  3         SUM(o.TOTAL_AMOUNT) AS TOTAL_PURCHASE_AMOUNT
  4  FROM Customer c
  5  JOIN Orders o
  6  ON c.ID = o.CUSTOMER_ID
  7  GROUP BY c.ID, c.NAME
  8  ORDER BY TOTAL_PURCHASE_AMOUNT DESC;

CUSTOMER_ID CUSTOMER_NAME        TOTAL_PURCHASE_AMOUNT
----------- -------------------- ---------------------
          1 Renu                                55000
          2 Arjun                               25000
          4 Rahul                                2500
          3 Priya                                1598
          5 Keerthana                             650
```

4. Identify Best-Selling Products

```
SQL> SELECT p.PRODUCT_ID,
  2         p.PRODUCT_NAME,
  3         SUM(od.QUANTITY) AS TOTAL_QUANTITY_SOLD
  4  FROM Product p
  5  JOIN Order_Details od
  6  ON p.PRODUCT_ID = od.PRODUCT_ID
  7  GROUP BY p.PRODUCT_ID, p.PRODUCT_NAME
  8  ORDER BY TOTAL_QUANTITY_SOLD DESC;

PRODUCT_ID PRODUCT_NAME
---------- --------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
       103 T-Shirt
                 2

       102 Smartphone
                 1

       105 Python Programming Book
                 1


PRODUCT_ID PRODUCT_NAME
---------- --------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
       104 Mixer Grinder
                 1

       101 Laptop
                 1
```

4.1 Product Sales Amount

```
SQL> SELECT p.PRODUCT_ID,
  2         p.PRODUCT_NAME,
  3         SUM(od.QUANTITY) AS TOTAL_QUANTITY_SOLD,
  4         SUM(od.QUANTITY * od.UNIT_PRICE) AS TOTAL_SALES_AMOUNT
  5  FROM Product p
  6  JOIN Order_Details od
  7  ON p.PRODUCT_ID = od.PRODUCT_ID
  8  GROUP BY p.PRODUCT_ID, p.PRODUCT_NAME
  9  ORDER BY TOTAL_SALES_AMOUNT DESC;

PRODUCT_ID PRODUCT_NAME
---------- --------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES_AMOUNT
------------------- ------------------
       101 Laptop
                   1              55000

       102 Smartphone
                   1              25000

       104 Mixer Grinder
                   1               2500


PRODUCT_ID PRODUCT_NAME
---------- --------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES_AMOUNT
------------------- ------------------
       103 T-Shirt
                   2               1598

       105 Python Programming Book
                   1                650
```

5. Category-wise Sales Analysis

```
SQL> SELECT c.CATEGORY_ID,
  2         c.CATEGORY_NAME,
  3         SUM(od.QUANTITY) AS TOTAL_QUANTITY_SOLD,
  4         SUM(od.QUANTITY * od.UNIT_PRICE) AS TOTAL_CATEGORY_SALES
  5  FROM Category c
  6  JOIN Product p
  7  ON c.CATEGORY_ID = p.CATEGORY_ID
  8  JOIN Order_Details od
  9  ON p.PRODUCT_ID = od.PRODUCT_ID
 10  GROUP BY c.CATEGORY_ID, c.CATEGORY_NAME
 11  ORDER BY TOTAL_CATEGORY_SALES DESC;

CATEGORY_ID CATEGORY_NAME                  TOTAL_QUANTITY_SOLD
----------- ------------------------------ -------------------
TOTAL_CATEGORY_SALES
--------------------
          1 Electronics                                      2
              80000

          3 Home Appliances                                  1
               2500

          2 Fashion                                          2
               1598


CATEGORY_ID CATEGORY_NAME                  TOTAL_QUANTITY_SOLD
----------- ------------------------------ -------------------
TOTAL_CATEGORY_SALES
--------------------
          4 Books                                            1
                650
```

6. Combined Business Sales Report

```
SQL> SELECT c.ID AS CUSTOMER_ID,
  2         c.NAME AS CUSTOMER_NAME,
  3         o.ORDER_ID,
  4         p.PRODUCT_NAME,
  5         cat.CATEGORY_NAME,
  6         od.QUANTITY,
  7         od.UNIT_PRICE,
  8         (od.QUANTITY * od.UNIT_PRICE) AS AMOUNT_GENERATED
  9  FROM Customer c
 10  JOIN Orders o
 11  ON c.ID = o.CUSTOMER_ID
 12  JOIN Order_Details od
 13  ON o.ORDER_ID = od.ORDER_ID
 14  JOIN Product p
 15  ON od.PRODUCT_ID = p.PRODUCT_ID
 16  JOIN Category cat
 17  ON p.CATEGORY_ID = cat.CATEGORY_ID
 18  ORDER BY o.ORDER_ID;

CUSTOMER_ID CUSTOMER_NAME          ORDER_ID
----------- -------------------- ----------
PRODUCT_NAME
--------------------------------------------------
CATEGORY_NAME                  QUANTITY UNIT_PRICE AMOUNT_GENERATED
------------------------------ ---------- ---------- ----------------
          1 Renu                          401
Laptop
Electronics                               1      55000            55000

          2 Arjun                         402
Smartphone
Electronics                               1      25000            25000

CUSTOMER_ID CUSTOMER_NAME          ORDER_ID
----------- -------------------- ----------
PRODUCT_NAME
--------------------------------------------------
CATEGORY_NAME                  QUANTITY UNIT_PRICE AMOUNT_GENERATED
------------------------------ ---------- ---------- ----------------

          3 Priya                         403
T-Shirt
Fashion                                   2        799             1598

          4 Rahul                         404
Mixer Grinder

CUSTOMER_ID CUSTOMER_NAME          ORDER_ID
----------- -------------------- ----------
PRODUCT_NAME
--------------------------------------------------
CATEGORY_NAME                  QUANTITY UNIT_PRICE AMOUNT_GENERATED
------------------------------ ---------- ---------- ----------------
Home Appliances                           1       2500             2500

          5 Keerthana                     405
Python Programming Book
Books                                     1        650              650
```